

Méthode d'analyse des données en grandes dimensions et apprentissage automatique pour la mécanique (MAD)

CM2 - Réseaux de neurones pour la classification et images

Felipe ROCHA

`felipe.figueredo-rocha@u-pec.fr`

dépot git : <https://github.com/felipefr/mad/> (CC BY-NC-SA 4.0)

Faculté des Sciences et Technologie, Univ. Paris-Est Créteil Val de Marne

(23/09/26)

Distribution Anaconda

- 1 Télécharger Anaconda depuis <https://www.anaconda.com>
- 2 Installer Spyder ou JupyterLab via Navigator (ou Terminal).

Commandes d'installation (Anaconda Prompt ou Terminal) :

```
# Installation de PyTorch  
conda install pytorch cpuonly -c pytorch  
# Installation de SciPy et Matplotlib  
conda install -c anaconda scipy  
conda install -c conda-forge matplotlib
```

Introduction à PyTorch et Manipulation de Tenseurs

Qu'est-ce que PyTorch ?

Une bibliothèque de Machine Learning majeure proposant :

- 1 Calcul tensoriel accéléré sur GPU.
- 2 Calcul automatique de gradients (autograd).

```
import torch

a = torch.tensor([[0., 2., 4.], [3., 4., 2.]]) #
      (2, 3)
b = torch.zeros(3, 3, 3)
      # (3, 3, 3)
c = torch.linspace(0, 1, 10)
      (10)

% Redimensionnement et conversion
res_c = c.view(1, 10)
x_np = a.numpy()           % Conversion vers
      NumPy
```



<https://docs.pytorch.org/docs/2.14/>

- **Principe** : Le calcul de gradients s'effectue via la différenciation automatique en construisant un graphe de calcul orienté.
- **Activation du graphe** : Défini avec `requires_grad=True` sur le tenseur d'entrée.
- **Noeuds du graphe** : Chaque tenseur contient :
 - `data` : La valeur brute du tenseur.
 - `grad` : Le gradient accumulé.
 - `grad_fn` : Référence vers la fonction de rétropropagation (ex: `MulBackward0`, `PowBackward0`).

Rétropropagation (backward)

Les gradients sont calculés à l'aide de la dérivation en chaîne (*chain rule*).

Implémentation Pratique des Gradients dans PyTorch

```
import torch
from torch.autograd import grad

x = torch.linspace(0., 1., 10, requires_grad=True)  # Initie le graphe
y = 2 * x
z = y ** 2

# Calcul du gradient de z par rapport a x
# retain_graph=True permet de préserver le graphe en memoire pour d'autres calculs
dz_dx = grad(z, x, torch.ones_like(z), retain_graph=True)

# Detachement du graphe (tres important pour le tracage ou la conversion vers NumPy)
x_np = x.detach().numpy()
dz_dx_np = dz_dx.detach().numpy()

# Visualisation simple du gradient
import matplotlib.pyplot as plt
plt.plot(x_np, dz_dx_np)
```

- **Contexte** : Pour les fonctions vectorielles (ex: relations forces-déplacements en mécanique), nous calculons la matrice jacobienne des dérivées partielles.

```
# Exemple d'operation lineaire y = A @ x
A = torch.tensor([[1., 2.], [3., 4.]])
x = torch.tensor([[2.], [1.]], requires_grad=True)
y = torch.matmul(A, x)

# Methode automatique standard de PyTorch pour calculer
  le jacobien
func_y = lambda x : torch.matmul(A, x)
jacobien = torch.autograd.functional.jacobian(func_y, x)
```

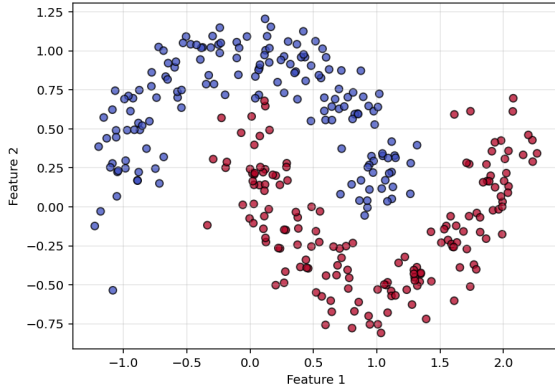
- **Optimisation diagonale** : Si l'opération est diagonale (ex: $y_i = x_i^2$), on exploite cela pour accélérer le calcul en passant un vecteur de uns dans `grad()`.

Notebook Pytorch basique

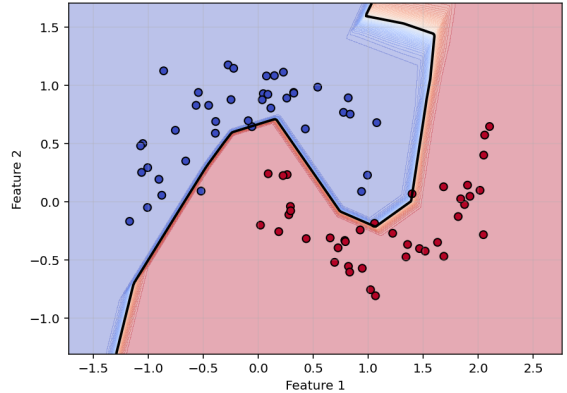
Notebook MLP pour la fonction sinus (Pytorch)

Classification

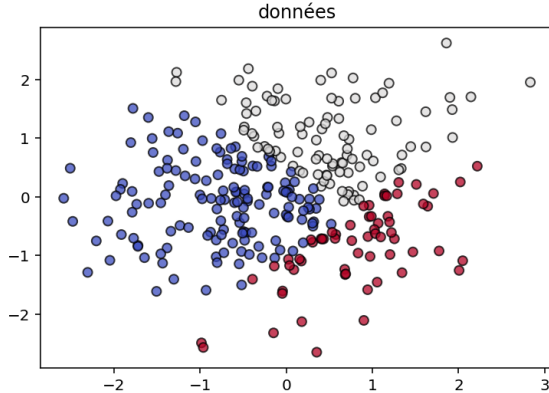
Dataset : deux classes



Frontière de décision (Accuracy = 95.00%)



Classification



Classification : Fonction Perte (Loss Function)

- Softmax function ($\mathbb{R}^n \rightarrow \mathbb{R}^n$):

$$[\text{SoftMax}(\underline{z})]_c = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}} \quad (1)$$

- Cross Entropy:

$$L_{CE} = - \sum_c y_c \log(\hat{p}_c), \quad (2)$$

tels que $y_c = \begin{cases} 1 & \text{si } c = \text{vrai classe} \\ 0 & \text{cas contraire} \end{cases}$, $\hat{p}_c \geq 0$ et $\sum_{c=1}^C \hat{p}_c = 1$ (sortie de la fonction SoftMax).

<https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html>

Classification Binaire: Fonction Perte (Loss Function)

- Binary Cross Entropy:

$$L_{BCE} = - \sum_c y_c \log(\hat{p}_c) + (1 - y_c) \log(1 - \hat{p}_c), \quad (3)$$

$\hat{p}_c$ étant la sortie d'une sigmoïd.

<https://docs.pytorch.org/docs/2.14/generated/torch.nn.BCELoss.html>

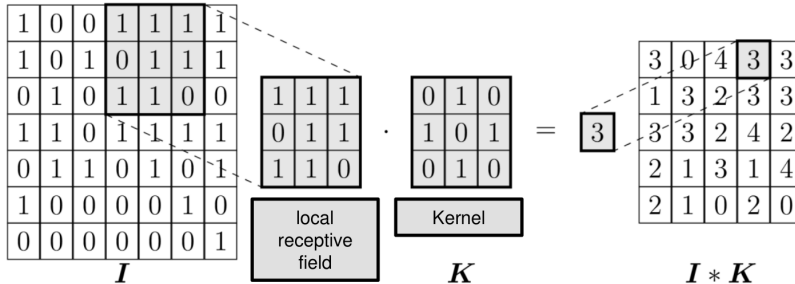
Script MLP Classification 3 classes

Script MLP Classification 2 classes (Moon)

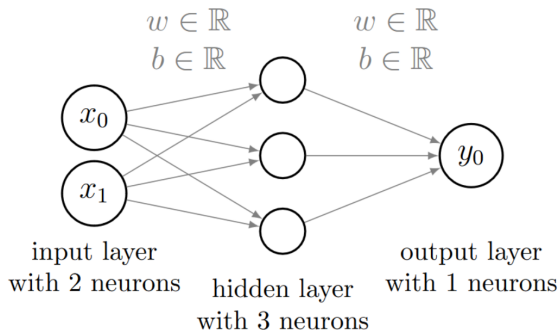
CNN : Convolutional Neural Networks

Opération Filtre Convolutif $\underline{\underline{O}} = \underline{\underline{I}} \star \underline{\underline{K}}$:

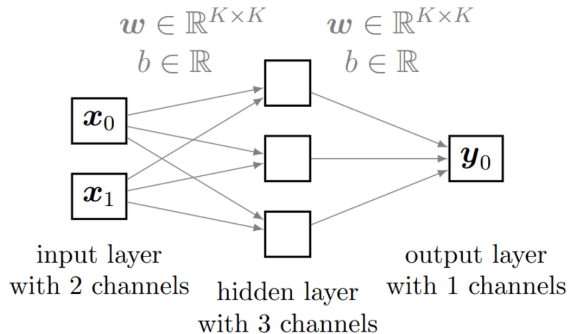
$$O_{kl} = \sum_{i,j} I_{(i+k)(j+l)} K_{ij} \quad (4)$$



MLP vs CNN



Fully connected neural network



Convolutional neural network

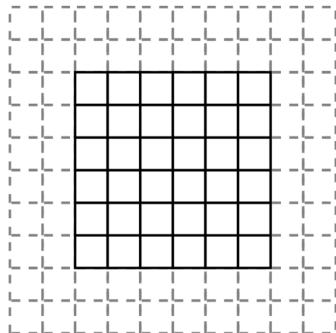
CNN : Couches Pooling

- Types : MaxPooling, AveragePooling
- Paramètres : Taille Kernel (K), Stride (S), Padding (P)
- Formule :

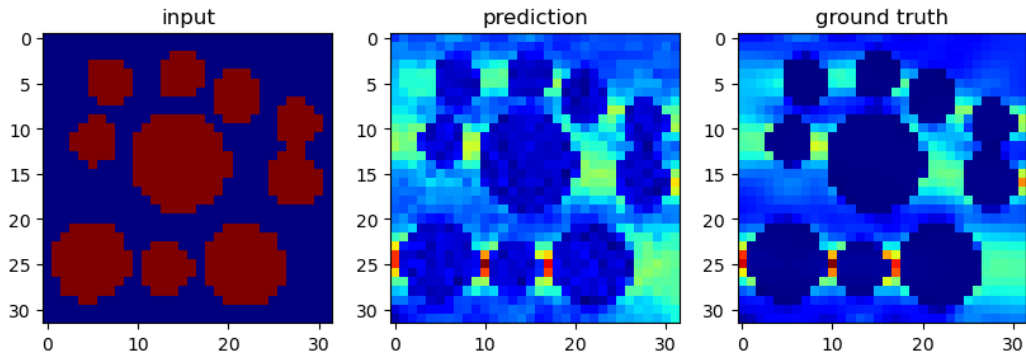
$$O = \left\lfloor \frac{I + 2P - K}{S} \right\rfloor \quad (5)$$

1	4	3	4	1
1	2	4	3	3
1	2	3	4	1
1	3	3	1	1
3	3	1	1	0

4	4	4	4
2	4	4	4
3	3	4	4
3	3	3	1



Application CNN : Prediction des champs de contraintes



Application CNN : Prediction des champs de contraintes

